

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SUBISH A G (192312139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Subish A G

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Ans: A hospital must assign **3 doctors (D1, D2, D3)** to **3 shifts (Morning, Afternoon, Night)** while satisfying all given constraints.

Variables

- Doctors: **D1, D2, D3**

Domain

- Morning (M), Afternoon (A), Night (N)**

Constraints

- D1 $\neq$ Night**
- D2 must work before D3** (Morning < Afternoon < Night)
- Only one doctor per shift**
- D3 $\neq$ Morning**
- Each doctor is assigned exactly one shift**

Constraint	Priority	Reason
One doctor per shift	High	Prevents duplicate assignments
Each doctor gets one shift	High	Ensures complete schedule
D1 cannot work Night	Medium	Restricts D1's domain
D3 cannot work Morning	Medium	Restricts D3's domain
D2 before D3	High	Determines valid order

Step 1: Assign D1

Try **Morning**

Valid

Current Assignment

Doctor	Shift
--------	-------

D1	Morning
----	---------

D2	—
----	---

D3	—
----	---

Step 2: Assign D2

Morning is already occupied.

Try **Afternoon**

Valid

Current Assignment

Doctor Shift

D1 Morning

D2 Afternoon

D3 —

Step 3: Assign D3

Morning - Not allowed

Afternoon - Already occupied

Try **Night**

Constraint Check

- D3 $\neq$ Morning
- D2 before D3 (Afternoon < Night)
- One doctor per shift

Accepted.

Constraint Checking

Constraint	Status
D1 cannot work Night	Satisfied
D2 before D3	Afternoon < Night
One doctor per shift	Satisfied
D3 cannot work Morning	Satisfied
Every doctor assigned once	Satisfied

No backtracking is required because the first valid path satisfies all constraints.

Final Valid Schedule

Shift Doctor

Morning **D1**

Afternoon **D2**

Night **D3**

The **Backtracking Search** algorithm assigns one doctor at a time and checks all constraints after each assignment. If any constraint is violated, it backtracks to the previous assignment and tries another option. In this problem, the first sequence satisfies all constraints, so no backtracking is needed.

The obtained schedule is valid because:

- **D1** is not assigned to the Night shift.
- **D2** is scheduled before **D3**.
- Each shift has exactly one doctor.
- **D3** is not assigned to the Morning shift.
- Every doctor is assigned exactly one shift.

Final Answer

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Night	D3

2. Ans: A robot must move from the **Start (S)** to the **Goal (G)** in a **5 × 5 grid**.

Constraints

- Grid size = **5 × 5**
- Allowed movements:
 - Up
 - Down
 - Left
 - Right
- Cost of each move = **1**
- Robot cannot move through obstacles.
- Manhattan Distance is given as the heuristic (used in informed search algorithms).

Constraint	Analysis
Move only in four directions	Valid moves: Up, Down, Left, Right
Cost per move = 1	Every edge has equal cost

Obstacles cannot be crossed

Invalid nodes are ignored

Reach Goal

BFS guarantees the shortest path in an unweighted grid

Example Grid:

```
S . X . .  
. . X . .  
. . . . .  
X X . X .  
. . . . G
```

S = Start

G = Goal

X = Obstacle

BFS Node Expansion

Step Expanded Node Queue after Expansion

1	S(0,0)	(0,1), (1,0)
2	(0,1)	(1,0), (1,1)
3	(1,0)	(1,1), (2,0)
4	(1,1)	(2,0), (2,1)
5	(2,0)	(2,1)
6	(2,1)	(2,2)
7	(2,2)	(2,3), (3,2)
8	(2,3)	(3,2), (2,4), (1,3)
9	(3,2)	(2,4), (1,3), (4,2)
10	(4,2)	(4,3), (4,1)
11	(4,3)	(4,4)
12	G(4,4)	Goal Reached

Optimal Path

S

↓
(1,0)
↓
(2,0)
→
(2,1)
→
(2,2)
↓
(3,2)
↓
(4,2)
→
(4,3)
→
G

Cost Calculation

Each move costs 1.

Number of moves = 8

Total Cost = 8

- BFS explores nodes **level by level**.
- It uses a **FIFO Queue**.
- Since every move has the same cost, BFS always finds the **shortest path**.

Obstacles are skipped during expansion.

BFS is the appropriate algorithm because:

- All moves have equal cost.
- It guarantees the shortest path.
- It systematically explores all reachable nodes until the goal is found.

3. Ans: The rescue robot must move from the **Start (S)** to the **Goal (G)** in a disaster-affected building while minimizing travel cost and avoiding dangerous areas. Since the environment is only partially observable, the robot has incomplete knowledge and must update its map as it explores.

Constraints and Their Impact

Constraint	Effect on Decision Making
Move only Up, Down, Left, Right	Limits possible actions at each step.
Obstacles cannot be crossed	Robot must search for alternative routes.
Limited sensing (adjacent cells only)	Robot cannot plan the complete path initially and must explore dynamically.
Cost per move = 1	Every movement increases the path cost.
Risky zone additional cost = 2	Robot avoids risky zones unless necessary to reduce total cost.
Dynamic knowledge update	The robot continuously updates its map and replans when new information is discovered.

Appropriate Search Strategy

Online Search Agent with Uniform Cost Search (UCS)

Justification

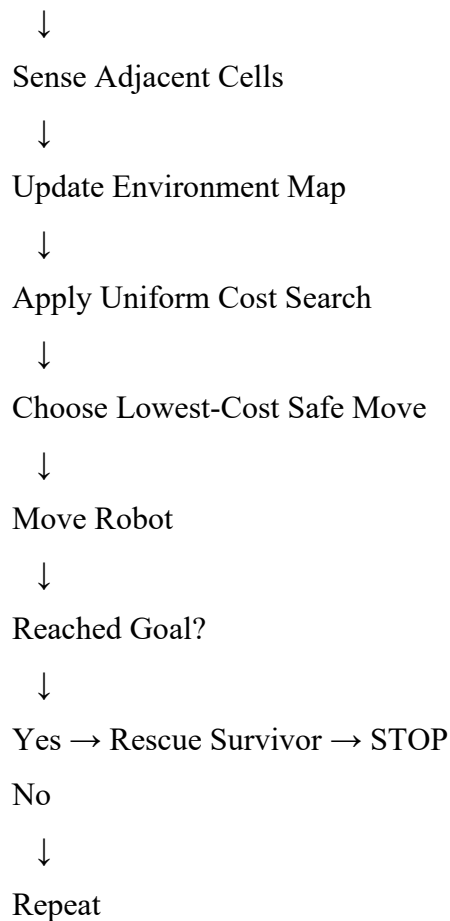
- The environment is **partially observable**.
- The robot learns about obstacles only while moving.
- UCS always expands the **lowest cumulative cost** path, considering both movement cost and risky-zone penalties.
- Online Search enables dynamic replanning whenever new obstacles are detected.

Algorithm Steps

1. Start at **S**.
2. Sense adjacent cells (Up, Down, Left, Right).
3. Update the internal map with observed obstacles and risky zones.
4. Compute the lowest-cost path using **Uniform Cost Search**.
5. Move to the lowest-cost neighboring cell.
6. Repeat sensing and updating after every move.
7. Continue until the robot reaches **Goal (G)**.

Flow of the Algorithm

START



1. Intelligent Agent

The rescue robot is an **Intelligent Agent** because it:

- Perceives the environment through sensors.
- Makes decisions autonomously.
- Acts using actuators (movement).
- Continuously updates its knowledge.

2. Rationality

The robot behaves **rationally** by:

- Selecting the path with the **minimum total cost**.
- Avoiding risky areas whenever possible.
- Replanning when new obstacles are detected.
- Achieving the goal efficiently.

3. Environment Type

Property	Type
Observability	Partially Observable
Determinism	Deterministic
Dynamics	Dynamic
State Space	Sequential
Agents	Single-Agent
Knowledge	Unknown environment (learned online)

The **Online Search Agent combined with Uniform Cost Search** is the most suitable approach because:

- It works effectively in **partially observable environments**.
- It dynamically updates the map as new information becomes available.
- It always chooses the **minimum-cost path**, considering both movement and risky-zone costs.
- It avoids blocked paths and adapts to changing conditions.
- It ensures efficient and safe navigation while reaching the survivor.